# Tabela hash: índice de palavras

## Configuração 40: E1.1, H4, C2 e A4

Enthony Araujo de Oliveira
Douglas Patrick Cardozo Freitas

Estrutura de Dados II
Professor: Kennedy Reurison Lopes

# Objetivo e roteiro

## Objetivo do trabalho

Implementar uma tabela hash própria e usá-la para consultar palavras de um texto.

1. Configuração e organização do projeto.
2. Normalização, conversão, função hash e colisões.
3. Operações e decisões da implementação.
4. Testes de correção e análise experimental.
5. Demonstração no terminal, com entradas e resultados esperados.

# Nossa configuração: nº 40

| Código | Escolha | Significado no projeto |
|---|---|---|
| E1.1 | Vetor fixo | Capacidade escolhida ao criar, sem crescimento. |
| H4 | Dobra | Soma de grupos decimais de quatro dígitos. |
| C2 | Sondagem linear | Em colisão, procura a próxima posição. |
| A4 | Índice de palavras | Palavra normalizada como chave, frequência e linhas como valor. |

**Implementação própria em C11**

Usamos vetores, strings, leitura de arquivo e temporização. Não usamos uma tabela hash pronta.

# O que a aplicação A4 guarda

**Exemplo de texto**

```
Linha 1:  Hash hash!
Linha 2:  HASH e tabela.
```

A mesma palavra pode aparecer várias vezes, inclusive com caixa diferente.

**Depois da normalização**

| Palavra | Frequência | Linhas |
|---|---|---|
| hash | 3 | 1, 2 |
| e | 1 | 2 |
| tabela | 1 | 2 |

**Frequência e linhas são informações diferentes**

A frequência conta cada ocorrência. O vetor de linhas registra cada linha uma única vez durante a leitura sequencial.

# Organização do código

| Arquivo | Responsabilidade |
|---|---|
| `src/tabela.h` | Tipos e declarações das operações. |
| `src/tabela.c` | Conversão, dobra, vetor, sondagem, operações e exibição. |
| `src/aplicacao.c` | Normalização e indexação do arquivo de texto. |
| `src/main.c` | Menu, entrada do usuário e abertura do arquivo. |
| `src/experimentos.c` | Medições experimentais. Correção pelo menu. |

**Fluxo:** o menu chama a aplicação, que normaliza as palavras e usa as operações da tabela.

Os cabeçalhos permitem que os arquivos compartilhem os mesmos tipos e funções.

# Estruturas usadas na tabela

```
typedef struct {
    char chave[TAM_PALAVRA];
    int frequencia;
    int *linhas;
    size_t quantidade_linhas;
} Palavra;

typedef struct {
    Estado estado;
    Palavra palavra;
} Posicao;
```

- `chave`: palavra de até 255 bytes, mais o terminador da string.
- `linhas`: endereço do vetor de inteiros com as linhas do texto.
- Cada posição contém um registro e um estado: vazio, ocupado ou removido.

# Normalização das palavras

- Convertemos letras para minúsculas.
- Mapeamos os acentos portugueses tratados no código para a letra base.
- Na leitura do arquivo, espaços e pontuação separam as palavras.
- O menu espera uma palavra por vez e aplica a mesma normalização.

| Entrada | Chave usada |
|---|---|
| `Hash, HASH` | `hash` |
| `Colisão,` | `colisao` |
| `guarda-chuva no arquivo` | `guarda e chuva` |

**Limites da regra**

Tokens com mais de 255 bytes são rejeitados. O tratamento de acentos é simples, sem suporte completo a todas as formas Unicode.

# Da palavra para um inteiro

```c
static uint64_t chave_para_inteiro(const char *palavra)
{
    uint64_t numero = 0;
    for (size_t indice = 0; palavra[indice] != '\0'; indice++) {
        unsigned char byte = (unsigned char)palavra[indice];
        numero = numero * 131 + byte;
    }
    return numero;
}
```

- Cada byte influencia o resultado, sempre na mesma ordem.
- Exemplo: `hash` produz **235481250**.
- `uint64_t` usa 64 bits sem sinal, com aritmética módulo $2^{64}$.
- Essa conversão prepara a chave textual. A função atribuída continua sendo a dobra H4.

# H4: cálculo por dobra

```c
size_t hash_indice(const char *palavra, size_t capacidade)
{
    uint64_t numero = chave_para_inteiro(palavra);
    uint64_t soma = 0;
    while (numero > 0) {
        uint64_t grupo = numero % 10000;
        soma = soma + grupo;
        numero = numero / 10000;
    }
    return (size_t)(soma % capacidade);
}
```

Para `hash`: 235481250 se divide em 2, 3548 e 1250.

$$\text{soma} = 2 + 3548 + 1250 = 4800, \qquad h(\texttt{hash}) = 4800 \bmod m.$$

Com $m = 2$, o índice é 0. O módulo garante um resultado em $[0, m-1]$.

# E1.1: capacidade fixa e estados

- Capacidade padrão: $m = 64$. Podemos escolher outro valor antes de iniciar.
- `calloc` reserva o vetor inicial. O programa não o redimensiona.
- "Estático", na convenção da atividade, significa capacidade fixa durante o uso.

| Estado | Como interpretar |
|---|---|
| `VAZIO` | Nunca ocupado desde a criação. Uma busca pode parar aqui. |
| `OCUPADO` | Há uma palavra válida para comparar. |
| `REMOVIDO` | Um registro foi excluído. A busca deve continuar. |

O vetor de linhas de uma palavra pode crescer. Isso não aumenta as posições da tabela hash.

# C2: colisão e sondagem linear

$$p_i(k) = (h(k) + i) \bmod m, \qquad i = 0, 1, \ldots, m-1.$$

Com $m = 2$, `hash` e `tabela` têm índice inicial 0.

| Palavra | Índice inicial | Posição ocupada | Motivo |
|---|---|---|---|
| `hash` | 0 | 0 | Primeira posição livre. |
| `tabela` | 0 | 1 | A posição 0 já estava ocupada. |

**O módulo permite voltar ao início**

Se a posição inicial for a última do vetor, a próxima tentativa usa a posição 0. Após m tentativas, todas as posições foram visitadas.

# Inserção: decisões do algoritmo

1. Procurar a chave para impedir uma duplicata, inclusive depois de um removido.
2. Calcular o índice inicial com H4.
3. Percorrer o vetor pela sondagem linear.
4. Usar a primeira posição vazia ou removida.
5. Registrar palavra e frequência, incrementar n e contar a colisão aplicável.

## Dois resultados importantes

**Duplicata:** retorna `RES_DUPLICADA`, sem substituir o valor.

**Sem posição disponível:** retorna `RES_CHEIA`, sem crescer o vetor.

# Busca: como o código percorre o vetor

```
size_t indice = (indice_inicial + tentativa) % tabela->capacidade;
Posicao *posicao = &tabela->posicoes[indice];
if (posicao->estado == VAZIO) {
    break;
}
if (posicao->estado == OCUPADO) {
    if (strcmp(posicao->palavra.chave, chave) == 0) {
        return indice;
    }
}
```

Trecho dentro do laço de sondagem. `strcmp(...)=0` significa strings iguais. Se não encontrar a chave, a função retorna `tabela->capacidade`. O laço visita no máximo m posições.

# Remoção: por que manter REMOVIDO

```
size_t indice = localizar(tabela, chave);
if (indice == tabela->capacidade) {
    return RES_AUSENTE;
}
free(tabela->posicoes[indice].palavra.linhas);
tabela->posicoes[indice].palavra = (Palavra){0};
tabela->posicoes[indice].estado = REMOVIDO;
tabela->quantidade--;
return RES_OK;
```

- Após remover `hash` da posição 0, `tabela` ainda está na posição 1.
- Se a posição 0 virasse `VAZIO`, a busca pararia antes de encontrar `tabela`.
- `REMOVIDO` preserva o caminho e permite reutilizar a posição depois.

# Atualização e listagem

```
size_t indice = localizar(tabela, chave);
if (indice == tabela->capacidade) {
    return RES_AUSENTE;
}
tabela->posicoes[indice].palavra.frequencia = frequencia;
return RES_OK;
```

- Exemplo: atualizar `tabela` de frequência 2 para 7 mantém a mesma chave.
- A atualização mantém as linhas já registradas e não cria outro registro.
- A listagem percorre o vetor e imprime somente posições ocupadas.
- A exibição da estrutura também mostra vazios, removidos, índice inicial e contadores.

# Indexação: palavra nova ou repetida

```
if (palavra != NULL) {
    palavra->frequencia++;
    tabela_adicionar_linha(palavra, numero_linha);
    ocorrencias_aceitas++;
}
```

- Primeiro, buscamos a palavra normalizada na tabela.
- Se ainda não existe, tentamos inseri-la com frequência zero.
- Com um registro disponível, somamos a ocorrência e guardamos a linha.
- Palavra já existente recebe novas ocorrências mesmo com a tabela cheia.

# Contadores, capacidade e custo

- $m$: quantidade fixa de posições do vetor.
- $n$: quantidade atual de palavras distintas armazenadas.
- $\alpha = n/m$: fator de carga, isto é, a ocupação da tabela.
- Uma colisão é contada quando uma nova chave aceita encontra sua posição inicial ocupada por outra.
- Duplicatas e atualizações não aumentam esse contador. Remover reduz n, mas não apaga o histórico de colisões.

## Custo das operações

A sondagem pode visitar até m posições: pior caso $O(m)$, com chaves de tamanho limitado. A distribuição das chaves e a ocupação influenciam o custo observado.

# Testes manuais: entradas e resultados

| Entrada / caso | Esperado e obtido na execução |
|---|---|
| Buscar em tabela vazia | Ausente. OK. |
| Inserir `hash`=1 e buscar | Frequência 1. OK. |
| Buscar chave inexistente | Ausente. OK. |
| Inserir `hash` novamente | Chave duplicada. OK. |
| Inserir `tabela` em colisão | Índices 0 e 0, uma colisão. OK. |
| Inserir terceira chave em m=2 | Tabela cheia. OK. |
| Remover `hash`, buscar `tabela` | `tabela` continua presente. OK. |
| Atualizar `tabela`=7 | Mesma chave, frequência 7 e n=1. OK. |

**Oito casos conferidos pelo menu do hashapp.**

Registro em `resultados/testes_manuais.md`: entradas, esperado, obtido e situação. A opção 6 exibe a estrutura. Redimensionamento não se aplica a E1.1.

# Análise experimental: metodologia

1. Usar $n = 100$, 1.000 e 10.000 registros, com $m = 2n$ e $\alpha = 0{,}5$.
2. Criar uma tabela nova em cada uma das **cinco repetições** por cenário.
3. Inserir as primeiras n chaves: `palavra0`, `palavra1`, etc.
4. Buscar as n chaves presentes e depois outras n chaves ausentes.
5. Medir inserção e buscas com `clock()`, além das colisões de inserção.

**Regra de medição**

Tempos de CPU em milissegundos para o lote inteiro. A média é a soma das cinco medições dividida por cinco. Geração das chaves e criação da tabela ficam fora da medição.

# Resultados: médias de cinco repetições

| n | m | $\alpha$ | Colisões | Inserção (ms) | Busca ok (ms) | Busca aus. (ms) |
|---|---|---|---|---|---|---|
| 100 | 200 | 0,50 | 18 | 0,023 | 0,012 | 0,036 |
| 1.000 | 2.000 | 0,50 | 304 | 0,354 | 0,144 | 0,281 |
| 10.000 | 20.000 | 0,50 | 4.953 | 3,606 | 1,595 | 3,302 |

- No maior cenário, 4.953 inserções encontraram o índice inicial ocupado.
- Capacidade suficiente: nenhuma necessidade de crescimento.
- Tempos representam lotes de n operações. Para uma operação, dividir por n.
- Os tempos podem variar quando os experimentos forem executados em outra máquina.

# Discussão dos resultados e limitações

- Mesmo com $\alpha = 0{,}5$, as colisões mudam com a capacidade e o conjunto de chaves.
- A dobra pode concentrar índices. A sondagem linear pode formar blocos de posições ocupadas.
- Uma busca ausente precisa encontrar um vazio ou percorrer a tabela, enquanto uma busca presente para ao encontrar a palavra.
- As chaves são sintéticas. Os tempos não representam a leitura e normalização de um texto completo.
- Mantivemos alfa constante: não medimos uma curva ao variar a ocupação.

**Precisão da medição**

Lotes pequenos têm precisão limitada pela resolução de `clock()`. Repetimos o mesmo conjunto para comparar as medições de forma consistente.

# Demonstração 1: compilar com GCC

Na pasta `tabela-hash`, executar:

```
gcc -std=c11 -Wall -Wextra -O2 \
  src/tabela.c src/aplicacao.c src/main.c -o hashapp

gcc -std=c11 -Wall -Wextra -O2 \
  src/tabela.c src/experimentos.c -o experimentos
```

- O primeiro comando gera `hashapp`, o programa interativo.
- O segundo gera `experimentos`, usado somente para medir desempenho.
- A barra no final da linha continua o comando no terminal.

# Demonstração 2: colisão, duplicata e tabela cheia

```
./hashapp 2
```

| Opção | Respostas seguintes | Resultado esperado |
|---|---|---|
| 1 | Palavra `hash`, frequência 1 | OK. |
| 2 | Palavra `hash` | Frequência 1. |
| 1 | Palavra `hash`, frequência 9 | Chave duplicada. |
| 1 | Palavra `tabela`, frequência 2 | OK, com colisão. |
| 6 | Nenhuma | m=2, n=2, uma colisão. |
| 1 | Palavra `extra`, frequência 1 | Tabela cheia. |

No campo **Opção**, digitar o número. A palavra só deve ser digitada quando aparecer **Palavra**.

# Demonstração 3: remoção, atualização e listagem

Continuar na mesma execução, após o exemplo anterior.

| Opção | Respostas seguintes | Resultado esperado |
|---|---|---|
| 4 | Palavra `hash` | OK. |
| 2 | Palavra `tabela` | Presente, frequência 2. |
| 2 | Palavra `hash` | Chave não encontrada. |
| 6 | Nenhuma | Posição 0: REMOVIDO. Posição 1: tabela. |
| 3 | Palavra `tabela`, frequência 7 | OK, mantém a chave. |
| 5 | Nenhuma | Uma palavra: tabela, frequência 7. |
| 0 | Nenhuma | Encerra o programa. |

**Linhas vazias são esperadas:** estes registros foram inseridos manualmente, sem arquivo de origem.

# Demonstração 4: indexar o texto de exemplo

```
./hashapp 64
```

1. Escolher a opção **7**.
2. Informar `dados/texto_exemplo.txt`.
3. Conferir: **55 ocorrências, 41 palavras distintas, 0 rejeitadas**.
4. Escolher **2** e consultar `Hash`: chave `hash`, frequência **3**, linhas **1, 2 e 5**.
5. Escolher **5** para listar e **6** para exibir a estrutura.
6. Escolher **0** para sair.

## O que este exemplo demonstra

Normalização, repetição de palavras, frequência, linhas de ocorrência e consulta por chave.

# Demonstração 5: executar os roteiros prontos

```
./hashapp 64 < dados/demo_config40.txt

./hashapp 32 < dados/demo_tabela_cheia.txt
```

- O símbolo < usa as linhas do arquivo como respostas do menu.
- Primeiro roteiro: indexação, duplicata, consultas, remoção, atualização e listagem.
- Segundo roteiro: capacidade 32 para 41 palavras distintas.
- Resultado do segundo: **44 ocorrências aceitas, 32 palavras distintas e 11 ocorrências rejeitadas**.

**Ocorrências não são palavras distintas**

As 11 rejeitadas incluem repetições de palavras que não conseguiram entrar. Palavras já armazenadas continuam acumulando frequência.

# Demonstração 6: medições e registro manual

```
./experimentos

./experimentos > resultados/experimentos_config40.md
```

- Correção: repetir os casos dos slides anteriores pelo menu.
- Conferir entradas, esperado e obtido em `resultados/testes_manuais.md`.
- Experimentos: conferir três linhas de resultados, com médias de cinco repetições.
- O símbolo > salva a saída, substituindo o conteúdo anterior do arquivo.
- Novas execuções podem produzir tempos diferentes, mesmo com as mesmas chaves.